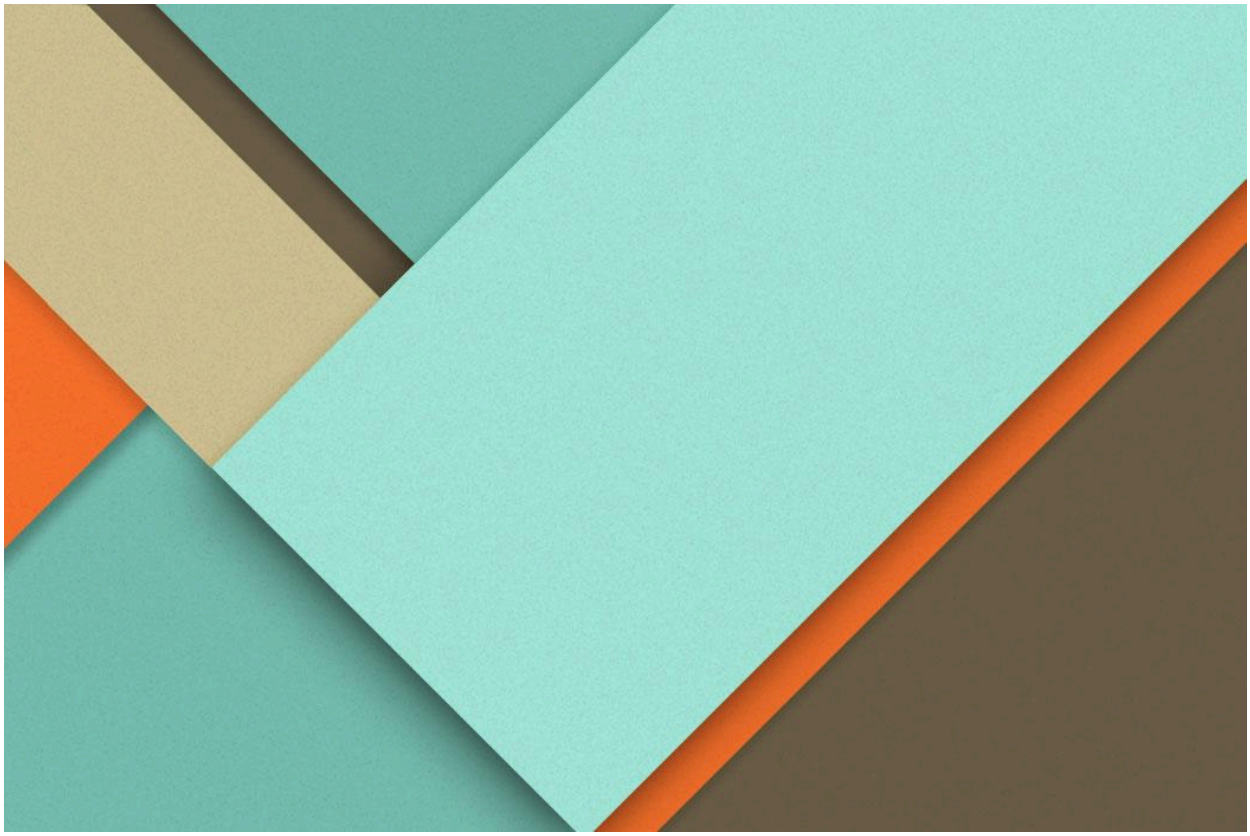

Projet : Gestion d'une agence immobilière

Programmation orientée objet (8PRO128-01)

Nathan Morasse | MORN16070400

Mathias Lavoie | LAVM19040400

29/04/26

Exercices **2**
Exercice 1 2
Exercice 2 2
Exercice 3 2
Exercice 4 2
Exercice 5 3
Exercice 6 3
Défis et solutions **4**
Encodage UTF-8 4
Problème 4
Solution 4
Dépendance circulaire 4
Problème 4
Solution 4
Tests et validations **5**
RealEstate 5
Person 5
Contract 6
Autres 6
Réflexion & Améliorations futures **7**

Exercices

Exercice 1

Les classes de base RealEstate (BienImmobilier), Person (Personne) et Contract (Contrat) ont été définies avec les attributs et méthodes indiqués dans l'énoncé. Chaque classe possède un identifiant unique généré automatiquement par un compteur statique. RealEstate expose une méthode virtuelle display() (afficherDetails) pour présenter les informations du bien. Person stocke les coordonnées ainsi qu'un vecteur de contrats, et fournit une méthode display() également virtuelle. Dans le main, plusieurs instances de ces classes sont créées et leurs méthodes sont appelées pour vérifier leur bon fonctionnement.

Exercice 2

Les sous-classes House, Apartment et Land héritent de RealEstate et redéfinissent display() pour ajouter des détails spécifiques à chaque type de bien (par exemple Apartment peut référencer un locataire). Les sous-classes Client, Owner et Tenant héritent de Person. Owner enrichit la classe de base avec un vecteur de biens possédés et des méthodes de gestion de propriété. Tenant maintient une liste des biens loués. L'héritage est testé en instanciant des objets concrets et en invoquant leurs méthodes, y compris via des pointeurs de type RealEstate* ou Person*, ce qui active le polymorphisme.

Exercice 3

Les sous-classes de "RealEstate" et "Person" ont toutes leur propre version de la fonction "display" ce qui permet à une fonction comme "detailed_list" d'utiliser cette fonction peu importe le type passé en paramètre et d'avoir un affichage personnalisé pour chacune des classe.

Exercice 4

Les exceptions sont levées dans les méthodes critiques comme "sign" (contrat déjà signé ou sans bien) et "process" (contrat non signé ou transaction déjà effectuée). L'agence capture ces erreurs via des blocs try/catch dans create_contract et sign_contract, puis affiche un message et libère les ressources si nécessaire. Ce mécanisme empêche l'application de planter en cas d'opération invalide et permet de tester facilement les scénarios d'erreur.

Exercice 5

La classe interface (Objet permettant de faciliter l'affichage) utilise quelques fonctions template. Les fonctions "detailed_list" et "simple_list" peuvent prendre en paramètre les une liste d'objets ayant une fonction "display" comme les propriétés, personnes où contrats et les affiches dans la console sous forme de liste. La fonction "num_input" prend des types numérique et fait la validation des entrées numériques que l'utilisateur fait.

Exercice 6

Les associations entre les entités ont été mises en œuvre à l'aide de vecteurs de pointeurs. La classe Person (et par héritage Client, Owner, Tenant) détient un vector<Contract*> pour représenter le lien « un client peut avoir plusieurs contrats ». Réciproquement, Contract référence le bien concerné (RealEstate*), le client (Person*) et le propriétaire (Owner*), ce qui matérialise l'association entre BienImmobilier et Contrat. L'agrégation est centralisée dans la classe Agency qui regroupe l'ensemble des biens (vector<RealEstate*>) et des personnes (vector<Person*>) et assure leur cycle de vie (allocation dynamique, libération dans le destructeur). Les contrats et transactions sont également stockés dans des vecteurs dédiés au sein de l'agence, garantissant une gestion mémoire cohérente et conforme au diagramme de classes.

Défis et solutions

Encodage UTF-8

Problème

Lors de l'exécution, les caractères accentués français (comme "é", "à", "ê") s'affichent sous la forme de ⁇ dans la console Windows. Les chaînes de caractères étaient correctement écrites dans le code source, mais la console ne parvenait pas à les interpréter.

Solution

Nous avons changé l'encodage des fichiers sources de Windows-1252 vers UTF-8 avec signature (BOM) dans Visual Studio, et ajouter l'option de compilation /utf-8. Au début du main(), nous avons appelé SetConsoleOutputCP(CP_UTF8) et SetConsoleCP(CP_UTF8) pour que la console utilise l'UTF-8. Après ces modifications, tous les accents se sont affichés normalement.

Dépendance circulaire

Problème

Au départ, Client.h inclut Contract.h pour que le vecteur contracts puisse stocker des pointeurs de Contract. Plus tard, lorsqu'il a fallu que Contract connaisse également le client signataire, l'inclusion de Client.h dans Contract.h créait une dépendance circulaire, empêchant la compilation.

Solution

Nous avons remplacé les #include superflus par des déclarations anticipés (forward declarations) dans les en-têtes. Chaque classe déclare simplement class Contract; ou class Client; avant de les utiliser comme pointeurs. Les #include complets n'apparaissent que dans les fichiers .cpp où l'on manipule effectivement les objets. Cela a éliminé la circularité tout en conservant un couplage faible.

Tests et validations

RealEstate

Test effectué	Résultat
Instancier un objet de la classe et s'assurer qu'il est bien créé	L'objet RealEstate est bien créé.
Appeler la fonction display() et s'assurer que les informations sont justes	Informations affichées correctement.
Appeler la fonction update_status() et s'assurer que les informations ont étés mis à jour	La fonction update_status() met bien à jour le statut du bien.
Tester l'héritage en instanciant des objets des sous-classes House, Apartment et Land puis appeler leurs méthodes	L'héritage fonctionne bien.
Tester le polymorphisme dans le main en utilisant un pointeur de type RealEstate pour appeler la méthode virtuelle sur les sous-classes	Les méthodes virtuelles produisent le bon résultat selon la classe utilisée.

Person

Test effectué	Résultat
Instancier un objet de la classe et s'assurer qu'il est bien créé	La personne a bien été créée
Appeler la fonction display() et s'assurer que les informations sont justes	La fonction display affiche bien les informations de l'objet
Tester l'héritage en instanciant des objets des sous-classes Client, Owner et Tenant puis appeler leurs méthodes	Les fonctions des sous-classes de Person fonctionnent comme voulu avec l'héritage

Contract

Test effectué	Résultat
Instancier un objet de la classe et s'assurer qu'il est bien créé	L'objet contrat est bien créé
Appeler la fonction display() et s'assurer que les informations sont justes	la fonction display() affiche les bonnes informations du contrat
Signer un contrat et s'assurer qu'une transaction est enregistrée pour le paiement	Une transaction est enregistrée lors de la signature du contrat
Provoquer des erreurs et vérifier que les exceptions sont bien capturés et gérées	Si un contrat a déjà le statut "Signé", et on tente de le resigner, une exception est lancée et gérée avec le message "Contrat déjà signé."

Autres

Test effectué	Résultat
Tester la fonction template avec différent types et s'assurer qu'elle fonctionne avec chacun	Les fonctions templates retournent les bonnes valeurs selon les types.

Réflexion & Améliorations futures

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet en C++ : héritage, polymorphisme, gestion mémoire avec pointeurs et destructeurs, exceptions, templates et associations. Nous avons appris à structurer une application console autour d'une classe centrale (l'agence) et à utiliser des forward déclarations pour éviter les dépendances circulaires. Pour l'avenir, plusieurs améliorations sont envisageables comme ajouter la persistance des données (sauvegarde/chargement dans un fichier), ou implémenter un système de suggestion de biens comme mentionné dans l'énoncé.